

A UART-Attached Coarse-Grained Reconfigurable Array as a Runtime-Reconfigurable Accelerator

Mattia Fait
Advanced Logic Design
University of Trento
Trento, Italy

Abstract—This report presents the logical design of a coarse-grained reconfigurable array (CGRA) implemented on an FPGA and driven from a host computer over a serial link. The fabric is a 4×4 mesh of 16-bit processing elements (PEs); the host reconfigures the array by loading one configuration word per PE, injects operands at the array edges, advances the datapath a chosen number of cycles and reads back the register file. A compact, checksum-protected UART protocol connects a host-side software stack—a C library and a UNIX-style command-line tool with a small configuration language—to the fabric. We describe the PE datapath, the mesh interconnect, the control finite-state machine, and the mapping of element-wise, reduction and weight-stationary systolic dataflows onto the array under the architectural constraint of a single register per PE with edge-only operand injection. We further show that the same constraint governs physical implementation: the PE’s combinational ALU chain is one step too deep for a naive one-step-per-clock cadence on the target Artix-7 part, which we close with a multicycle timing exception rather than by adding PE state, and quantify with a frequency search that reports, and lets us track, the limiting path at each iteration. A wire-level cost model identifies serial round trips, not processing elements, as what bounds the accelerator’s observed speed; acting on it—a fused execute transaction that returns only the registers a mapping taps, an operand placement that makes the prefix scan’s configuration data-independent, and the elision of all-zero matrix tiles—reduces measured round trips by $2\text{--}19\times$ and bytes by up to $6.7\times$, with every kernel’s results bit-identical and pre-existing bitstreams still supported.

Index Terms—CGRA, reconfigurable computing, systolic array, FPGA, dataflow, UART protocol.

I. INTRODUCTION

Coarse-grained reconfigurable arrays (CGRAs) occupy the middle ground between the flexibility of software and the efficiency of fixed-function hardware [1]. Rather than reconfiguring at the bit level, as an FPGA does, a CGRA exposes a grid of word-level processing elements (PEs) whose operations and interconnect are set by a compact configuration, allowing the same silicon to be re-purposed at run time for different kernels.

This project implements such a fabric on an FPGA and uses it as a *runtime-reconfigurable accelerator* attached to a host computer over a serial link. The design goals are threefold: (i) a small but expressive fabric whose behaviour is fully controlled, cycle by cycle, from the host; (ii) a robust host/device protocol that can be exercised and verified without the board; and (iii) a

UNIX-style software stack that lets a user describe and run accelerated kernels declaratively.

A single architectural decision runs through the whole system and gives the report its narrative spine: each PE owns exactly *one* register and external operands enter the mesh *only* at two edges. We trace the consequences of that decision from the datapath up to the dataflows a user can express and down to the bytes on the wire.

Concretely, this report contributes:

- a fully host-controlled 4×4 CGRA fabric whose every step is gated by a compact, checksum-protected UART protocol (Sections III–IV);
- a mapping of element-wise, reduction and weight-stationary systolic dataflows onto the array, with an explicit argument for *why* the single-register/edge-injection constraint admits some and excludes others (Section V);
- a layered host stack—a thin C library, a byte-accurate device model that serves as a golden reference, and a declarative command-line front end—that runs and is verified end to end without an FPGA (Sections VI, IX);
- a wire-level cost model that shows the accelerator’s observed speed is bounded by serial round trips rather than by the array (Section VII), and—acting on it—a fused execute transaction with a masked read-back, together with two mapping-level changes, that cut measured round trips by $2\text{--}19\times$ while leaving every kernel’s results bit-identical and older bitstreams supported (Sections IV–C, IX);
- a batch physical-implementation flow that closes timing on the target Artix-7 device with a multicycle exception rather than added PE state, and a frequency-search methodology that measures, and reports why, the fabric currently tops out where it does (Section VIII).

The remainder of this report is organised around the logical model of the system. Section II places the fabric in the CGRA and systolic literature. Section III describes the processing element and the mesh interconnect. Section IV presents the control finite-state machine and the UART protocol. Section V discusses how element-wise, reduction and systolic dataflows map onto the array, including the architectural constraint that shapes those mappings. Section VI outlines the host software stack, Section VII models its wire-level cost, Section VIII

closes timing on the target device and measures how much clock headroom the fabric has, and Section IX summarises the verification methodology and reports the measured wire cost against the model.

II. BACKGROUND AND RELATED WORK

A. The CGRA design space

Coarse-grained reconfigurable arrays trade the bit-level flexibility of an FPGA for word-level processing elements whose operation and interconnect are set by a compact configuration [1]. The design space is broad: PEs range from single ALUs to tightly coupled VLIW datapaths, as in ADRES [2]; interconnects vary from nearest-neighbour meshes to richer crossbars, as in MorphoSys [3], or to reconfigurable multi-hop links that let a value cross several PEs in one cycle, as in HyCUBE [6]; and configuration may be static for a whole kernel, streamed as a sequence of contexts, as in PipeRench [4], or specialised per-datapath and driven by a host core, as in DySER [5]. More recent fabrics such as Plasticine [7] specialise the PEs and memories around parallel patterns, and the mapping problem these fabrics share is itself a research area with its own modelling frameworks [8]. Across this space the recurring tension is the same: richer PEs and interconnect raise peak throughput but enlarge the configuration and the state that must be managed each cycle.

The fabric described here sits at the minimal end of that spectrum by design. It is a nearest-neighbour mesh of single-ALU, single-register PEs with a statically loaded, one-word-per-PE configuration. This minimality is not incidental: it is what lets the entire cycle-by-cycle behaviour of the array be exposed to, and driven from, a host over an ordinary serial link, and it is what makes an exact software model of the device tractable.

B. Systolic dataflow

The systolic principle—many identical cells, each doing a little work and passing data to a neighbour on every beat, so that memory bandwidth is amortised over a deep pipeline—dates to Kung and Leiserson [9], [10] and underpins modern matrix engines such as the tensor processing unit [11]. A canonical output-stationary array fuses, in each cell, a multiply with the accumulation of a partial sum arriving from one neighbour while activations flow orthogonally, so each cell holds two independent live values. Which operand a cell holds still—weights, activations, or the partial sum—is the standard taxonomy of dataflows for such arrays [12], [13], normally chosen to minimise data movement and hence energy. Section V shows why the single register per PE of this fabric precludes the output-stationary fusion and selects a *weight-stationary* streaming dataflow instead—the same trade-off as in the literature, but here forced by an explicit architectural constraint rather than chosen for energy.

C. Position of this work

This report does not propose a new microarchitecture. Its contribution is a complete, honest, end-to-end *logical* treatment

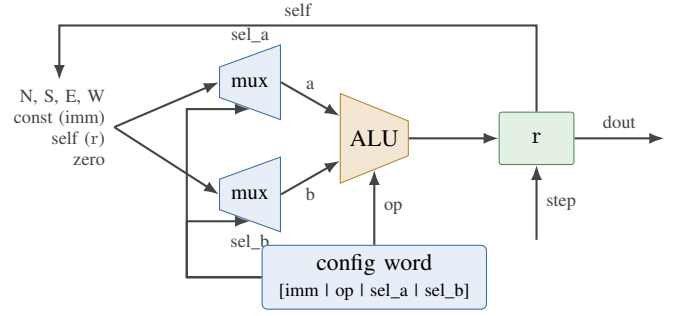


Fig. 1. Processing-element datapath: two operand muxes select from the four neighbours, the immediate, the self register or zero; the ALU result is latched by a single step-gated register.

of a deliberately small CGRA: a fully host-controlled fabric, a self-recovering byte-level protocol, a byte-accurate device model that doubles as a golden reference, and a UNIX-style software stack—all connected by a single narrow interface and all verifiable without an FPGA. The value is pedagogical and methodological: it makes visible, in one coherent system, how an architectural constraint propagates all the way up to the dataflows a user can express.

III. FABRIC ARCHITECTURE

The fabric is a $\text{ROWS} \times \text{COLS} = 4 \times 4$ mesh of identical processing elements operating on 16-bit signed words. All PEs share a single clock domain and advance synchronously; a global *step* enable, asserted by the controller, gates every PE register so that the host advances the whole array a deterministic number of cycles per command.

A. Processing element

Figure 1 shows the PE datapath. Each PE has two operand multiplexers that independently select one of seven sources: the registered output of a north, south, east or west neighbour; a 16-bit immediate carried in the configuration word; the PE's own register (a feedback path); or a constant zero. The selected operands drive an arithmetic-logic unit; its result is latched into the single output register when *step* is asserted.

The ALU implements sixteen operations selected by a 4-bit opcode: a no-op that holds the register; a pass-through; addition, subtraction and multiplication (the low 16 bits of the product); a multiply-accumulate and a simple accumulate, both folding into the register; the bitwise AND/OR /XOR; logical/arithmetic shifts; signed maximum, minimum and absolute value; and a constant load. Multiplication and multiply-accumulate keep the low half of the result, so arithmetic wraps modulo 2^{16} , consistent with a 16-bit datapath.

A single 32-bit configuration word fully specifies a PE: the low sixteen bits are the immediate, followed by the 4-bit opcode and the two 3-bit operand selects. One such word per PE constitutes a complete array configuration.

B. Mesh interconnect and operand injection

Figure 2 shows the interconnect. Internally, each PE reads the *registered* outputs of its four neighbours, so data advances

TABLE I
UART COMMAND SET. EXEC IS THE VERSION-3 ADDITION ANALYSED IN
SECTION IV-C; THE REST ARE VERSION 2.

Cmd	Payload	Reply
ID	—	signature, version, geometry
CFG	16 words + ck	ACK / NACK
WR	8 words + ck	ACK / NACK
RUN	step count N	ACK (after N steps)
RD	—	16 registers + ck
RST	—	ACK (clears registers)
EXEC	N , flags, tap mask, 8 words + ck	masked registers + ck + status

by at most one hop per step. External operands enter only at two edges: the north input port feeds row 0 (one value per column) and the west input port feeds column 0 (one value per row). The south and east edges read a constant zero. All sixteen PE registers are exposed for read-back.

This edge-only injection, combined with the single register per PE, is the defining architectural constraint of the fabric: interior PEs receive fresh host data only through their neighbours, so any computation that uses the whole array must route operands inward from the edges.

IV. CONTROL AND PROTOCOL

The host drives the fabric through a byte-oriented UART protocol interpreted by a control finite-state machine (FSM). The protocol is deliberately minimal: single-byte commands, each with a fixed payload and reply, so that the same message stream can be produced and checked by both the hardware and a software model.

A. Command set

Table I lists the commands. Configuration and input transfers carry an 8-bit additive checksum; the device rejects a corrupted transfer with a negative acknowledgement, and appends a checksum to the register read-back. The identification reply reports the array geometry, so the host can adapt at run time rather than assuming fixed dimensions.

B. A transaction on the wire

To make the framing concrete, consider writing the edge inputs west = (1, 0, 0, 0) and north = (5, 0, 0, 0) with the WR command. The host sends the command byte 0x03 followed by the four west then four north values as 16-bit little-endian words, and finally the additive checksum:

```
03 0100 0000 0000 0000 0500 0000 0000 0000
    06
```

The sixteen payload bytes sum to $1 + 5 = 6$, so the trailing checksum is 0x06; the device recomputes the same sum, finds it matches, and replies with a single ACK byte (0x79). A corrupted payload would sum differently and draw a NACK (0x1F) instead, on which the host retransmits. Every multi-byte transfer follows this shape—command, fixed payload, one checksum byte, one-byte acknowledgement—which is what

lets the same stream be produced and checked identically by the hardware FSM and the software model.

C. The fused execute transaction

The six commands above are complete but not economical, and the reason is visible in how a kernel actually uses them. Every element-wise chunk issues the same triple—write the edge operands, run, read the registers—so each chunk costs three round trips, and the read returns all sixteen registers even when the mapping produced four results. Section VII shows this is where the accelerator’s observed speed goes.

EXEC collapses that triple into one transaction (Figure 3). Its payload is the WR operand block preceded by three fields: the step count, a flags byte, and a 16-bit *tap mask* in which bit i selects PE register i . The device latches the operands, optionally clears the datapath (flags bit 0, a folded RST), steps the array, and streams back only the masked registers, a data checksum and a status byte. A four-lane element-wise chunk therefore ships eight result bytes instead of thirty-two, in one turnaround instead of three.

Two properties keep the fused command as recoverable as the ones it replaces. First, the reply length is a function of the tap mask *the host itself sent* and of nothing else, so the host can always frame the reply—even when the payload checksum fails, in which case the device returns the current registers with a NACK status and does *not* step the array. This is what lets a fused transaction be rejected without desynchronising the link, which a variable-length or early-aborting reply would not. Second, the fused reset makes the transaction idempotent, and the host retransmits after a rejection exactly when that flag is set; without it a failure resynchronises the device, as for a bare RUN. The distinction is not a convention but a consequence of what the command does to state: with the reset, the outcome depends only on the message; without it, the accumulator being built up is precisely what a replay would corrupt.

Since the identification reply carries the protocol version, a host that meets an older bitstream simply falls back to the unfused sequence. Both paths are held to the same known-answer suite (Section IX).

D. Controller state machine

Figure 4 shows the control FSM. From the idle state the first received byte selects a branch. The short transactions (ID, RST and the rejection of an unknown command) reply immediately through a shared transmit handshake. The bulk transfers accumulate their payload, verify (or emit) a checksum in a dedicated check state, and then reply. The RUN branch asserts *step* once per remaining cycle before acknowledging.

Notably, EXEC adds no second machine. Its header state falls through into the *same* operand-payload state WR uses and the *same* step engine RUN uses; only the return targets differ, held in two small registers. The fused command is thus a different path through the existing states rather than a parallel implementation of them—which is what keeps one byte-accurate software model able to mirror both.

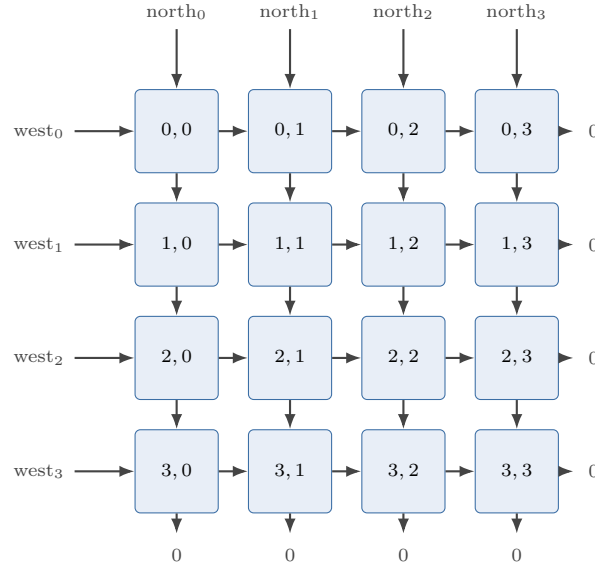


Fig. 2. The 4×4 mesh. Solid arrows are nearest-neighbour register connections; operands are injected only at the north edge (per column) and the west edge (per row), while the south and east edges read zero. This edge-only injection is the central constraint on dataflow mapping (Section V).

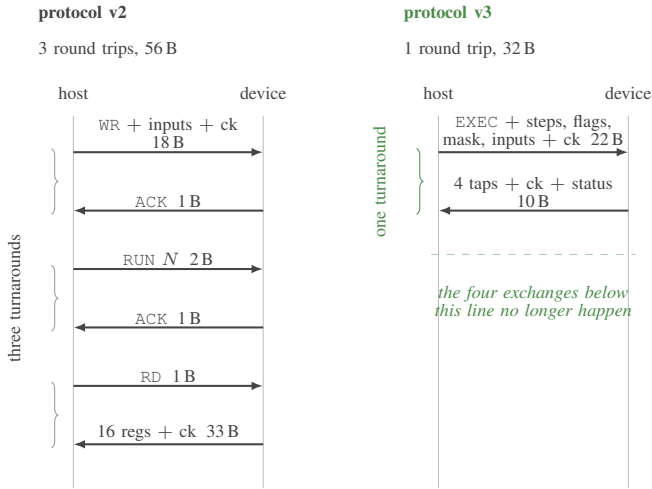


Fig. 3. One element-wise chunk on the wire. The unfused sequence (left) spends three turnarounds and returns all sixteen registers; EXEC (right) spends one and returns only the four registers the tap mask names, with the datapath clear folded into its flags byte. Because the reply’s length follows from the mask the host sent, a rejected payload still frames correctly.

A watchdog timer guards every state that waits for payload bytes: if no byte arrives within a bounded interval the machine aborts back to idle. Together with the checksum, this makes the link self-recovering—a property the host side mirrors by retransmitting rejected transfers and resynchronising after a failed run.

V. DATAFLOW MAPPING

A configuration is only useful if the array can be fed correctly. This section describes how three classes of computation map onto the fabric, and why the architectural constraint of Section III admits some mappings and excludes others.

A. Element-wise and reduction modes

The *diagonal* mapping computes element-wise binary operations. Operand a is injected on the north edge and operand b on the west edge; PEs above the diagonal pass the a values southward and those below pass the b values eastward, so that each diagonal PE receives one (a, b) pair and computes one output. Four elements are produced per transaction, one per diagonal PE.

The *reduction* mapping accumulates a stream into a single PE using the multiply-accumulate operation: at each step the host injects one pair on the north and west ports of PE(0, 0), which folds the product into its register. After the stream is exhausted the register holds the dot product.

B. Weight-stationary systolic matrix-vector

A matrix-vector product $y = Ax$ uses all sixteen PEs. Because interior PEs cannot receive host data directly, the vector x is held *stationary* in the PE immediates while the matrix is *streamed*: row 0 multiplies each incoming matrix element by its column weight, and rows 1–3 form a delay line that shifts the partial products southward. Figure 5 shows the resulting space-time skew: after $R = \text{ROWS}$ steps, PE row $R-1-i$ holds the products for output row i , which the host sums. Larger matrices are tiled over 4×4 blocks.

C. Why weight-stationary

A classical output-stationary systolic array (as in a tensor processing unit [9], [11]) fuses, in each PE, a multiply with the addition of a partial sum flowing from a neighbour, while a second stream of activations flows orthogonally. That requires each PE to forward one value *and* retain a distinct accumulator—two independent live values. The present fabric provides a single register per PE, and external data enters only at two edges. Fused spatial accumulation is therefore not realisable here,

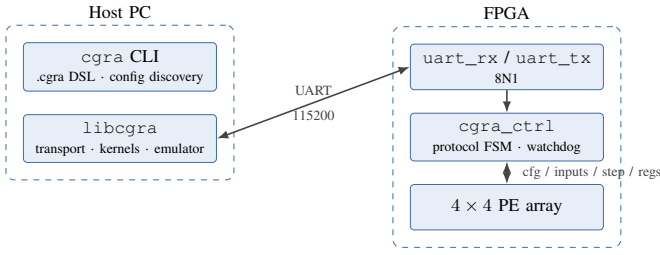


Fig. 6. Layered host/device architecture. The command-line tool compiles a declarative configuration language into per-PE words and drives the library, which reaches the fabric over UART or an in-process emulator.

the protocol version from a single identification handshake and selects, per chunk, either the fused command or the three-transaction sequence, so one binary drives both a current and an older bitstream. The same switch is what makes the optimisation measurable: because the device model can be told to report the older version and refuse the fused command, identical work can be run down both paths and the resulting transaction and byte counters compared directly (Section IX).

Above the library, a command-line tool provides high-level, UNIX-style access. A small declarative configuration language describes named *devices*, *modes* (array configurations, expressed either as a high-level pattern or as explicit per-PE placement), *pipelines* that chain modes, and input/output profiles. Configuration is discovered from standard locations and merged by name, following the drop-in convention used by system daemons, so that a user’s library of kernels can be organised across files and overridden selectively. Vectors flow through the tool via standard input and output, allowing kernels to compose in pipelines with ordinary UNIX utilities.

This separation—a thin, robust library and a declarative front end—keeps the protocol in one place while making the fabric approachable: a user reconfigures the accelerator by naming a mode, not by hand-assembling configuration words.

VII. COST AND PERFORMANCE MODEL

Because the fabric is driven entirely over the serial link, its externally observed performance is set not by the number of PEs but by the number and size of the protocol transactions a kernel issues. This section gives a first-order analytical model of that cost, derived from the protocol of Section IV; it deliberately ignores the negligible on-device compute time (a RUN of N steps is N steps at 100 MHz, dwarfed by the serial exchange). Section IX then checks the model against counters taken from the running stack.

A. Per-command wire cost

Every transaction is a round trip: the host sends a command and its payload and waits for the reply. Table II lists the byte counts that follow directly from the payload and reply sizes of Table I, for the 4×4 , 16-bit fabric.

The decisive line is the last one. An element-wise chunk under the unfused sequence costs $WR + RUN + RD = 56$ bytes over three round trips; the same chunk as one EXEC with $t = 4$

TABLE II
WIRE COST PER TRANSACTION (BYTES EACH WAY). t IS THE NUMBER OF REGISTERS SELECTED BY AN EXEC TAP MASK.

Cmd	TX	RX	Note
ID	1	5	signature + geometry
CFG	66	1	16×4 words + cmd + ck
WR	18	1	8×2 words + cmd + ck
RUN	2	1	cmd + step count
RD	1	33	16×2 regs + ck
RST	1	1	clears registers
EXEC	22	$2t + 2$	WR payload + 3 header bytes; reply + ck + status

costs 32 bytes over one. The byte saving comes almost entirely from not returning twelve registers the mapping never wrote.

B. Per-kernel transaction count

Composing these primitives as the software stack does (Section VI) gives the cost of each kernel. Writing $\lceil \cdot \rceil$ for the ceiling and $T_C = \lceil N/C \rceil$, $T_R = \lceil M/R \rceil$ for the number of column and row tiles, the transaction counts are:

- *Element-wise*, n elements (R lanes per pass): one CFG, then $\lceil n/R \rceil$ passes. Unfused, each pass is $WR + RUN + RD$; fused, each is a single EXEC. The count falls from $1 + 3\lceil n/R \rceil$ to $1 + \lceil n/R \rceil$.
- *Reduction* (dot product), n elements: the accumulator lives in one PE register, so this loop is one element per round trip by construction. Fusing the operand write into the step halves it, from $3 + 2n$ to $2 + n$. The mapping trades throughput for on-device accumulation, and no protocol change alters that—only the constant.
- *Systolic* $M \times N$ matrix-vector: one CFG per column tile and, per tile pair, R wavefront steps. Unfused that is one RST, R $WR + RUN$ rounds and one RD ($10T_R$ transactions per column tile); fused, the first step absorbs the reset and the last carries a full-array read-back, giving $4T_R$.

Two further reductions are algorithmic rather than protocol-level, and both follow from noticing that a transaction was carrying no information:

- The prefix scan originally held each block’s elements in the PE immediates, which made the configuration data-dependent and forced a 67-byte CFG every C elements—more wire traffic than the data itself. Injecting the elements on the north edge instead makes the configuration data-independent, so it is loaded once for the whole vector. Correctness is unchanged because C steps still flush the chain: PE $(0, c)$ settles at step $c + 1$.
- An all-zero $R \times C$ block of A contributes nothing to y whatever x is, so streaming it is pure wire cost. Skipping such blocks is exact, costs one 16-element scan per tile, and leaves dense matrices unaffected—but convolution’s Toeplitz matrix [15] is banded and mostly zeros, which is where the bulk of its saving comes from.

C. Latency model

At 115200 baud, 8N1, each byte occupies ten symbol times, so a byte costs $t_b = 10/115200 \approx 86.8 \mu\text{s}$. Modelling a transaction as the serial time of its two halves, an unfused element-wise round (56 bytes) takes about 4.9 ms for four results, i.e. roughly 1.2 ms per element, independent of what the ALU computes; the fused round (32 bytes) takes about 2.8 ms, or 0.70 ms per element.

That byte-counting model is, however, only a lower bound on what the round-trip count costs, and it is worth being explicit about why. A byte-rate model implicitly assumes the link is saturated, but each round trip also pays a turnaround that is not a function of length: the host must observe the reply before it can send the next command, and on the USB-serial bridge these boards use, the bridge coalesces bytes into USB frames on a latency timer [22], so a short reply is delayed by a fixed interval rather than by its size. Under such a per-turnaround cost the figure of merit is the *number* of round trips, not the bytes, and the two diverge: for the reduction and the dense matrix-vector, fusing costs a few percent more bytes while halving the turnarounds. Both quantities are therefore reported separately in Section IX rather than collapsed into one "speedup", and the byte figures are the conservative ones.

VIII. PHYSICAL IMPLEMENTATION AND TIMING CLOSURE

The preceding sections describe the fabric as a logical machine: registers, muxes and an FSM that behave identically in simulation and (this section shows) in silicon. Realising that machine at a useful clock frequency exposes one more consequence of the single-register-per-PE constraint that has run through the whole report: with no PE-internal pipeline stage, the entire ALU chain of Fig. 1 must resolve combinationally within one step, and that chain turns out to be too deep for the target part at a naive one-step-per-clock cadence. This section describes the batch Vivado flow, the timing exception that closes the design without adding PE state, the frequency-search methodology used to quantify and track headroom, and a floorplanning experiment whose negative result is reported honestly.

A. Toolchain and target

The fabric is synthesised and implemented with Vivado driven entirely in batch mode (`vivado -mode batch`), so the physical-design flow is scriptable and diffable Tcl with no project files checked in—the same discipline the report already applies to GHDL simulation. The primary target is a Digilent Nexys 4 DDR (Xilinx Artix-7 xc7a100tcsq324-1 [17]); a Digilent Basys 3 (xc7a35tcsq236-1) is retained as a secondary, smaller part, useful precisely because it stresses the same timing exception on a device with a different DSP-column layout (Section VIII-E returns to why that difference matters).

Every stage shares one Tcl procedure that applies the clock period, the false-pathed asynchronous I/O, and the multicyle exception of Section VIII-B; only the pin-level, per-board constraints (package pins, I/O standards) live outside it. A

single period argument therefore drives synthesis-only checks, full implementation and the frequency search of Section VIII-C alike, and the RTL's multicyle generic (Section VIII-B) and the exception's cycle count are driven from that same argument so the two cannot silently disagree—a real risk in hand-maintained multicyle designs, where the netlist and the constraint are edited in different files. Three gates run in increasing order of cost: a synthesis-only pass that fails on any elaboration error or critical warning (seconds); a full implementation with a static-timing gate that fails unless both setup and hold slack are non-negative; and the frequency search itself. On both parts the RTL synthesises with zero errors and zero critical warnings, one DSP48E1 per PE (sixteen total, an exact one-to-one mapping) and no inferred latches; on the Nexys 4 DDR this is 16/240 (6.7%) of the device's DSP48E1 slices and 4.9k/63.4k (7.7%) of its Slice LUTs, essentially unchanged across the operating points of Table III.

B. The datapath is one step too deep for one clock

Synthesising the naive design—*step* pulsed every clock, exactly as Section IV describes it at the protocol level—and implementing it at the standard 100 MHz (10 ns) clock failed static timing by 1.8 ns of negative setup slack, simultaneously at 256 endpoints. Every failing path had the same shape: from one PE's output register, through the operand mux, the DSP48E1 multiplier, a carry-chain accumulate, and the sixteen-way opcode mux, into a neighbouring PE's output register—eleven logic levels and roughly 11.7 ns of combined logic and routing delay, i.e. precisely the chain of Fig. 1 with no register in the middle. The violation proved resistant to placement/routing effort alone: retiming together with the most performance-aggressive synthesis, placement and routing directives [20] recovered under 0.2 ns, confirming the wall is structural, not a placement accident.

Two disciplined fixes exist that do not touch the ISA-visible contract of Sections III–IV. One is to pipeline the multiply/accumulate internally, but that adds a PE-internal register—exactly the state the report's guiding constraint rules out, and a change the host-side emulator (Section VI, the golden reference of Section IX) would have to mirror cycle-for-cycle to stay byte-accurate. The other is purely temporal: tell the timing engine that this datapath is allowed more than one clock period per logical step, while the RTL keeps producing exactly one result per step. We chose the latter. The controller (Section IV) now asserts *step* on the first of every `G_STEP_DIV` clock cycles per logical array step—a small generic-controlled phase counter, default 2—and idles for the remaining cycles before decrementing the `RUN` step count; a matching Vivado multicyle-path exception (`G_STEP_DIV` cycles for setup, `G_STEP_DIV-1` for hold) is placed from every PE, configuration and edge-input register onto every PE result register, so the timing engine is told the same fact the controller enforces. Figure 7 shows the resulting cadence for `G_STEP_DIV = 3`: *step* launches the chain on cycle 0, the datapath has the full multicyle window to settle, and the register captures on the next rising edge that starts a new

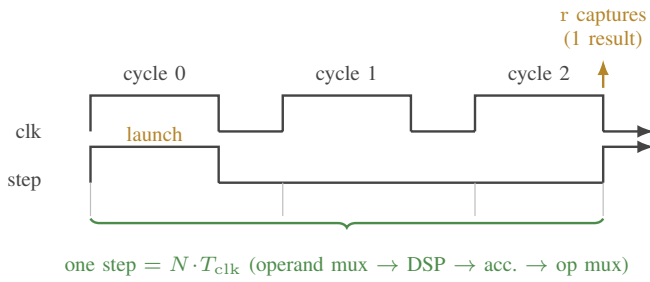


Fig. 7. Multicycle step cadence. *step* launches the ALU chain on the first of G_STEP_DIV clock periods (three shown); the register captures on the edge that starts the next step, so one step is always one result regardless of the cycle count spent settling.

TABLE III
MEASURED F_{max} ON THE NEXYS 4 DDR (ROUTE-ONLY SEARCH, CONSERVATIVE).

G_STEP_DIV	F_{max}	Step-rate	Limiting path
2	171.4 MHz	85.7 Msteps/s	datapath (11 levels)
3	192.8 MHz	64.3 Msteps/s	controller (8 levels)

step—one step, one result, independent of the cycle count spent getting there. We verified in RTL simulation, for several values of the factor, that the observed RUN/RD behaviour is bit-identical to the single-cycle case: the protocol and the emulator only ever observe the count of logical steps, never the physical clocks spent per step, so this is a case where a constraint that looks purely physical is really part of the same ISA-preservation argument that runs through the rest of the report. A multicycle exception is the standard way of telling a timing engine that a path is allowed more than one clock period [21], and Vivado provides it directly [19]; what is specific to this design is driving its cycle count from the same generic that paces the controller, so the two cannot drift apart.

C. Frequency search: how fast, and against what

A single pass/fail timing gate answers “does it close” but not “by how much, and against what.” We instead binary-search the clock period for the fastest one that still meets setup and hold, re-applying the timing exception (Section VIII-B) to the same synthesised netlist at each trial rather than re-synthesising, and report both the winning frequency and the limiting path’s two endpoints, logic-level count and whether it belongs to the multicycle datapath or to an ordinary single-cycle control path—so every run names a concrete next target instead of a bit. The search omits the placement-only optimisation pass to keep the reported number a conservative lower bound: a full implementation (as `make bit` runs it) only does at least as well.

Table III reports the two operating points measured on the Nexys 4 DDR. At the default factor of two, the design closes at 171.4 MHz, still limited by the datapath chain of Section VIII-B. Raising the factor to three moves the wall entirely off that chain (now comfortably inside its three-period

budget) and onto an eight-level, single-cycle path inside the protocol controller—a byte-assembly/checksum register chain—closing at 192.8 MHz, a 12.5 % increase. This is a genuine clock-versus-throughput trade, not free speed: each logical step still costs G_STEP_DIV physical clocks, so the step-rate that matters for an accelerator is F_{max}/G_STEP_DIV , which is *higher* at the smaller factor (Table III). Because Section VII already showed the serial link, not the array, sets the accelerator’s externally observed speed, the practical default is the factor that maximises step-rate; the higher-factor number is a statement of clock headroom, a ceiling rather than a throughput metric. The board’s certified operating clock remains 100 MHz at the default factor, where the static-timing gate reports 2.111 ns of setup slack and 0.048 ns of hold slack—comfortable margin, consistent with an F_{max} over 70 % higher.

D. Attacking the two named walls

Because the search names its wall rather than merely reporting a frequency, each operating point in Table III comes with a specific next target, and both have been acted on. Both changes are in the design and pass every functional check; neither has a measured frequency attached, because the machine on which they were made has no licensed implementation tool. They are reported here as implemented-and-pending rather than folded into the table, for the same reason the floorplanning result in Section VIII-E is reported as a negative: a number that has not been measured does not belong in a column of measurements.

The datapath wall is the multiply feeding a fabric carry chain. The multiply-accumulate was written as “truncate the product, then add the register”; it is now written as “add the register to the full-width product, then truncate”. The two are arithmetically identical—the low 16 bits of a sum depend only on the low 16 bits of its addends—but only the second is the $P = A \times B + C$ shape the DSP48E1 slice implements internally [18], so the accumulate need no longer be built from fabric logic hanging off the multiplier output. That is precisely the tail of the eleven-level chain of Section VIII-B.

The control wall is the read-back path. At higher multicycle factors the binding path ran from the register-index counter, through a sixteen-way multiplexer over the PE registers, into the checksum adder, all within one clock. The multiplexer’s output is now registered before it reaches the adder, so a clock carries the multiplexer *or* the adder and never both. This costs nothing in throughput: every consumer of that value reaches it through at least two states of the transmit handshake, and one serial byte is some 868 clocks at 100 MHz, so the pipeline stage is absorbed entirely. The one place needing care is the fused command’s masked read-back, which advances the index without transmitting when a register is not selected; that path takes an extra settle cycle so the registered value is never read one index behind.

E. Floorplanning: a negative result, honestly reported

Roughly half of the critical chain’s delay in Section VIII-B is routing rather than logic, which motivated an experiment:

constrain the sixteen PE instances (and their sixteen DSP48E1 multipliers) into one compact region of the die, so neighbour-to-neighbour nets stay short. The floorplan step queries the target part at run time for the clock region richest in DSP48 sites and constrains the array to it, rather than hand-picking site ranges; an earlier, hand-coded attempt using explicit DSP-column coordinates failed outright when retargeted from the Basys 3 to the Nexys 4 DDR, whose xc7a100t part has a different number of DSP columns than the xc7a35t—a concrete demonstration that physical, device-specific constraints are exactly the ones that do not port, and the reason the final version queries the part instead of encoding a layout.

Measured at the same clock period, floorplanning is neutral-to-negative for this design: constraining the array improved its internal wiring but stretched the wide array↔controller interface (the configuration, edge-operand and read-back buses spanning all sixteen PEs), moving the wall onto that interface and turning +0.10 ns of setup slack into −0.04 ns at the same period. The unguided placer already packs a design this small efficiently. We report this as a negative but informative result rather than omitting it: the technique is implemented, scripted and reusable, and the regime where it should help—a larger array, or one sharing the die with unrelated logic—is simply not the one this design occupies.

IX. VERIFICATION AND MEASURED WIRE COST

A. Verification

The design is verified at two levels that meet in the middle at the byte-level protocol. On the hardware side, VHDL testbenches driven through the UART interface (simulated with GHDL [16]) exercise the processing element in isolation, the full protocol (including checksum rejection, register read-back, and the fused transaction’s masked read-back, empty-mask and corrupted-payload cases), and the systolic matrix–vector mapping end to end. On the host side, the software model is used as a golden reference: it re-implements the PE arithmetic and the protocol FSM, so a discrepancy between the two implementations surfaces immediately.

Because the model is byte-accurate, every accelerated kernel is checked against an independent host computation without any hardware: element-wise arithmetic, the dot-product reduction, the tiled systolic matrix–vector product, prefix scans and convolution. Two deterministic fault-injecting variants of the model keep the recovery paths honest: one forces the retransmission path by rejecting the first configuration and the first operand-bearing transaction, and one reports protocol version 2 and refuses EXEC. The second is what makes the optimisation trustworthy rather than merely fast—the entire known-answer suite is run twice, once fused and once unfused, and the two must agree value for value. The same suite is reused as a bring-up check on real hardware once the board is programmed.

B. Measured wire cost

The cost model of Section VII can be checked directly, because the library counts transactions and bytes in each direction

TABLE IV
VERIFICATION SURFACE.

Level	Coverage
RTL (simulation)	PE ALU; full protocol with checksum; fused execute (masked / empty mask / corrupted payload); systolic matrix–vector
Model (host)	Protocol framing; PE arithmetic; all kernels vs. reference; retransmission
Cross-protocol	Every kernel run fused and unfused; results must be identical
Cross-check	Model is the golden reference for the RTL (shared semantics)

TABLE V
ROUND TRIPS AND BYTES FOR IDENTICAL WORK, BEFORE AND AFTER.
COUNTS ARE EXACT AND REPRODUCIBLE, NOT TIMINGS.

Workload	Round trips		Bytes	
	before	after	before	after
Element-wise, 20 480 elem.	15 380	5 141	288 060	165 186
Reduction, 20 480 elem.	41 020	20 541	452 620	493 586
Prefix scan, $n = 256$	320	66	8 000	2 121
Matrix–vector, 32×32	648	264	8 472	8 728
Convolution, $k = 5$, $n = 64$	2 736	144	34 800	5 168

and those counts are deterministic: they are measurements, not estimates, and they repeat exactly. Table V reports identical workloads before and after the three changes; Figure 8 plots the same data normalised per kernel.

A word on what the two columns compare. For the element-wise and reduction rows, “before” and “after” differ only in the protocol, so those rows isolate the fused transaction. The remaining three rows also include the mapping and zero-tile changes, which is the honest reading of what the work achieved but not of what fusion alone contributes. That can be separated too, by forcing the current stack down the unfused path: fusion by itself accounts for a 2.0–3.9× round-trip reduction on every kernel, and the rest of convolution’s factor—2 736 round trips down to 336 before fusion is even applied—is the zero tiles it stopped streaming.

Three effects are worth separating, because they have different reach. Fusion removes two of every three round trips for element-wise work and one of every two for the reduction; where the tap mask also shortens the reply it cuts bytes by 1.7×, and where it does not—the reduction and the dense matrix–vector, whose replies were already short or already needed the whole array—it costs 3–9 % more bytes for half the turnarounds, the trade Section VII anticipated. Loading the scan’s configuration once rather than per block removes a 67-byte transfer per four elements, which had outweighed the data. Skipping all-zero tiles is what makes convolution 19× cheaper in round trips: its Toeplitz matrix is banded, and the engine had been streaming its zeros.

One negative observation belongs here for the same reason the floorplanning result belongs in Section VIII-E. Measured end to end against the in-process model, the wall-clock

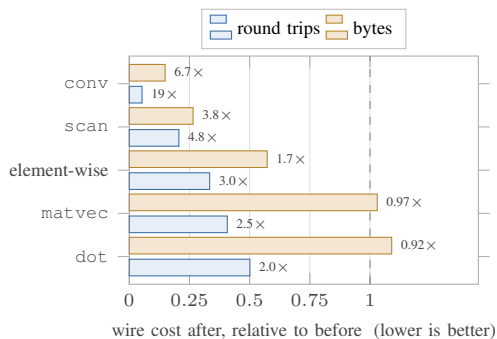


Fig. 8. Wire cost after the three changes, relative to before, per kernel (lower is better; the dashed rule is parity). Round trips fall for every kernel; bytes fall wherever a masked read-back or a skipped tile removes payload, and rise slightly where the fused header replaces a shorter command pair.

improvement is only 1.15–1.23 $\times$, far below the transaction ratios. That is not a failure of the optimisation but a property of the measurement: the model has no serial link, so what is being timed is the PE simulation, not the wire. On a real 115200-baud link the serial time exceeds the host-side compute by roughly four orders of magnitude, which is precisely why the byte and round-trip counts—not the emulated stopwatch—are the performance figures reported here.

X. CONCLUSION

We have described the logical design of a UART-attached CGRA: a 4×4 mesh of 16-bit processing elements, a checksum-protected control protocol with a compact finite-state machine, and a layered host stack that reconfigures the fabric declaratively and verifies it end to end without hardware. The recurring theme is a single architectural constraint—one register per PE with edge-only operand injection—which rules out fused spatial accumulation and thereby selects weight-stationary streaming as the natural systolic dataflow for the fabric, and which reappears in Section VIII as the reason the ALU chain has no internal pipeline stage to shorten it: closing timing on the target Artix-7 device therefore had to be a purely temporal fix—a multicycle exception paired with a matching cadence in the controller—rather than one that added PE state. Measured on the Nexys 4 DDR, that fix closes the 100 MHz operating clock with comfortable margin and a frequency search shows over 70% of additional headroom above it, while an automatic floorplanning experiment shows, honestly, that packing the array is not the right lever for a design this small on a device this size.

That headroom also frames what the accelerator’s speed actually depends on. Because the wire-level model of Section VII identifies serial round trips, not processing elements, as the binding cost, the productive optimisations were the ones that removed transactions rather than the ones that raised the clock. Three did so, and they illustrate three different kinds of waste: a protocol-level one, in which the write/run/read triple every chunk issued became a single fused transaction that also returns only the registers the mapping taps; a mapping-level

one, in which moving the prefix scan’s operands from the PE immediates to the array edge made its configuration data-independent and so loadable once instead of once per four elements; and an algorithmic one, in which tiles of a matrix that are entirely zero are no longer streamed at all. Together these cut round trips by 2–19 $\times$ and bytes by up to 6.7 $\times$, at no cost in fabric area and—because the identification handshake carries the protocol version—no loss of compatibility with an older bitstream. The measurement discipline mattered as much as the changes: the counts are deterministic and were taken on both protocol paths of the same binary, and the two cases where fusing costs slightly *more* bytes are reported rather than averaged away.

Future work follows from the same constraint that shapes the rest. Adding a second per-PE register (a dedicated partial-sum path) or widening the operand injection would unlock output-stationary systolic mappings [12], [13] and, with them, in-fabric matrix–matrix products and spatial finite-impulse-response filters; Section VIII shows this would also relax the timing picture, since a pipelined multiply-accumulate is precisely the change that would shorten the combinational chain that currently sets the multicycle datapath’s budget. On the link, the next step is the one the fused transaction stops short of: batching several chunks into a single transaction. That is bounded not by the protocol but by the controller, which today cannot receive and transmit concurrently, so a burst would need either a receive FIFO or a small result buffer—a concrete, scoped piece of hardware work rather than an open question. On the implementation side, the frequency search already names the next two targets precisely—the datapath’s operand-mux/DSP/accumulate chain at the default multicycle factor, and the controller’s checksum path at higher factors—and the two changes made against them are in the tree awaiting a measurement pass. Together these directions preserve the report’s guiding principle—that a single, explicit architectural constraint should account for the system’s behaviour at every layer, physical implementation included.

REFERENCES

- [1] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. Han, S. Yin, and S. Wei, “A Survey of Coarse-Grained Reconfigurable Architecture and Design: Taxonomy, Challenges, and Applications,” *ACM Comput. Surv.*, vol. 52, no. 6, pp. 118:1–118:39, 2019.
- [2] B. Mei, S. Vernalde, D. Verkest, H. De Man, and R. Lauwereins, “ADRES: An Architecture with Tightly Coupled VLIW Processor and Coarse-Grained Reconfigurable Matrix,” in *Proc. Int. Conf. Field Programmable Logic and Application (FPL)*, LNCS 2778, 2003, pp. 61–70.
- [3] H. Singh, M.-H. Lee, G. Lu, F. J. Kurdahi, N. Bagherzadeh, and E. M. Chaves Filho, “MorphoSys: An Integrated Reconfigurable System for Data-Parallel and Computation-Intensive Applications,” *IEEE Trans. Comput.*, vol. 49, no. 5, pp. 465–481, 2000.
- [4] S. C. Goldstein, H. Schmit, M. Moe, M. Budiu, S. Cadambi, R. R. Taylor, and R. Laufer, “PipeRench: A Coprocessor for Streaming Multimedia Acceleration,” in *Proc. 26th Int. Symp. Computer Architecture (ISCA)*, 1999, pp. 28–39.
- [5] V. Govindaraju, C.-H. Ho, and K. Sankaralingam, “Dynamically Specialized Datapaths for Energy Efficient Computing,” in *Proc. 17th Int. Symp. High Performance Computer Architecture (HPCA)*, 2011, pp. 503–514.
- [6] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh, “HyCUBE: A CGRA with Reconfigurable Single-Cycle Multi-Hop Interconnect,” in *Proc. 54th Design Automation Conf. (DAC)*, 2017, pp. 45:1–45:6.

- [7] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A Reconfigurable Architecture for Parallel Patterns," in *Proc. 44th Int. Symp. Computer Architecture (ISCA)*, 2017, pp. 389–402.
- [8] S. A. Chin, N. Sakamoto, A. Rui, J. Zhao, J. H. Kim, Y. Hara-Azumi, and J. Anderson, "CGRA-ME: A Unified Framework for CGRA Modelling and Exploration," in *Proc. 28th Int. Conf. Application-Specific Systems, Architectures and Processors (ASAP)*, 2017, pp. 184–189.
- [9] H. T. Kung, "Why Systolic Architectures?," *Computer*, vol. 15, no. 1, pp. 37–46, 1982.
- [10] H. T. Kung and C. E. Leiserson, "Systolic Arrays (for VLSI)," in *Sparse Matrix Proceedings 1978*, SIAM, 1979, pp. 256–282.
- [11] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, "In-Datacenter Performance Analysis of a Tensor Processing Unit," in *Proc. 44th Int. Symp. Computer Architecture (ISCA)*, 2017, pp. 1–12.
- [12] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [13] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [14] G. E. Blelloch, "Prefix Sums and Their Applications," School of Computer Science, Carnegie Mellon Univ., Tech. Rep. CMU-CS-90-190, 1990.
- [15] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.
- [16] T. Gingold, "GHDL: a VHDL 2008/93/87 simulator," <https://github.com/ghdl/ghdl> (accessed Aug. 10, 2026).
- [17] AMD/Xilinx, "7 Series FPGAs Data Sheet: Overview," DS180 (v2.6.1), 2020.
- [18] AMD/Xilinx, "7 Series DSP48E1 Slice User Guide," UG479 (v1.10), 2018.
- [19] AMD/Xilinx, "Vivado Design Suite User Guide: Using Constraints," UG903 (v2022.2), 2022.
- [20] AMD/Xilinx, "UltraFast Design Methodology Guide for FPGAs and SoCs," UG949 (v2022.2), 2022.
- [21] J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs: A Practical Approach*. New York, NY, USA: Springer, 2009.
- [22] Future Technology Devices International, "FT2232H Dual High Speed USB to Multipurpose UART/FIFO IC Datasheet," DS_FT2232H (v2.7), 2023.
- [23] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2013.